

More Attention

Advanced NLP M26

Manish Shrivastava + Aparajitha



September 3, 2026

Standard Attention

Scaled Dot-Product Attention

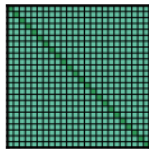
Disadvantages:

- Quadratic in time.
- Every token attends to every other.
- Fixed context window limits how big an input sequence can be.

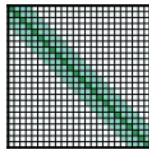


Sliding Attention

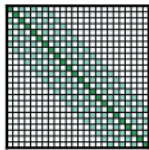
- Attends to neighbours in a window
- Linear in input size and window size.
- Stacking enables distant influence.



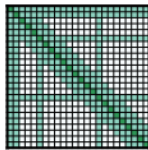
(a) Full n^2 attention



(b) Sliding window attention



(c) Dilated sliding window



(d) Global+sliding window



Sliding Attention

Limitations

- Long-range interaction requires information to traverse many stacked layers.
- Setting W too small can lose important information. Setting it too large beats the purpose.



Sliding Attention

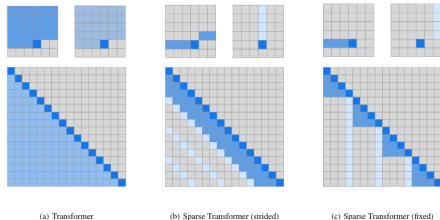
d	Embedding Dim	2048
n	Context Len / Window	2048
d_h	Q K V Dim	128
h, l	Heads, Layers	16
w	Attention window	128

Component	Calculation	MParams (per layer)	MParams	Memory (MB)
K	$d_h \times d \times l$	0.26	4	16
Q	$d_h \times d \times h \times l$	4	67	256
V	$2 \times d_h \times d \times l$	0.52	8	32
KV Cache	$w \times l \times 2 \times d_h$	0.03	1	4

4 bytes at FP32, 2 bytes at FP16/BF16, 1 byte at FP8/INT8

Sparse Attention

- Attends to a window of tokens and a stride.
- Aimed mainly at image processing.



Sparse Attention

d	Embedding Dim	2048
n	Context Len / Window	2048
d_h	Q K V Dim	128
h, l	Heads, Layers	16
w	Attention Window	128

Component	Calculation	MParams (per layer)	MParams	Memory (MB)
K	$d_h \times d \times l$	0.26	4	16
Q	$d_h \times d \times h \times l$	4	67	256
V	$2 \times d_h \times d \times l$	0.52	8	32
KV Cache	$w \times l \times 2 \times d_h$	0.02	1	4

4 bytes at FP32, 2 bytes at FP16/BF16, 1 byte at FP8/INT8

Sparse Attention

(DeepSeek)

- Attends to a selected few tokens (no window).
- “Selected few” are tokens with top-k attention scores in a single specialized head with very low precision.



Sparse Attention

(DeepSeek)

d	Embedding Dim	2048
n	Context Len / Window	2048
d_h	Q K V Dim	128
h, l	Heads, Layers	16
k	Selected tokens	64

Component	Calculation	MParams (per layer)	MParams	Memory (MB)
K	$d_h \times d \times l$	0.26	4	16
Q	$d_h \times d \times h \times l$	4	67	256
V	$2 \times d_h \times d \times l$	0.52	8	32
KV Cache	$k \times l \times 2 \times d_h$	0.01	0.5	2

4 bytes at FP32, 2 bytes at FP16/BF16, 1 byte at FP8/INT8

Sparse Attention

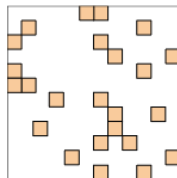
Limitations

- Fixed stride/block patterns assume regular structural patterns which aren't necessarily there.
- Non-contiguous memory access patterns are difficult to optimize on modern GPU memory.
- The DeepSeek approach has additional overhead to pick the top-k tokens.

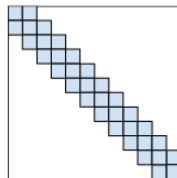


Hybrid Attention (BigBird)

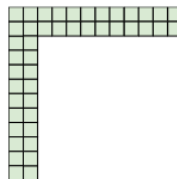
- Combination of a bunch of different methods, all $\mathcal{O}(N)$
- Allows for local and global context for tasks that need them both (like QA).



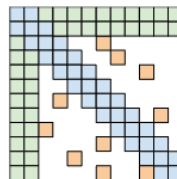
(a) Random attention



(b) Window attention



(c) Global Attention



(d) BIGBIRD

Hybrid Attention

d	Embedding Dim	2048
n	Context Len / Window	2048
d_h	Q K V Dim	128
h, l	Heads, Layers	16
g, w, r	Global, Window, Random tokens	64+128+64

Component	Calculation	MParams (per layer)	MParams	Memory (MB)
K	$d_h \times d \times l$	0.26	4	16
Q	$d_h \times d \times h \times l$	4	67	256
V	$2 \times d_h \times d \times l$	0.52	8	32
KV Cache	$(g + w + r) \times l \times 2 \times d_h$	0.06	2	4

4 bytes at FP32, 2 bytes at FP16/BF16, 1 byte at FP8/INT8

Hybrid Attention

Limitations

- Choosing which tokens should act as “global” tokens is task-dependent.
- Combining different attention patterns (sliding, random, global) makes parallelization tricky.



Linear Attention

- Since standard attention multiplies $\text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right)_{N \times N}$ and $V_{N \times d}$, it's $O(N^2)$ in time.
- The exponentiation in softmax can be eliminated in favour of some other "similarity" function.
- The similarity function can be chosen to be a kernel map, involving a pointwise function ϕ .
- Applying ϕ to the matrices is $O(Nd_k)$ in time, since kernel can be treated as a recurrent computation.

Standard (softmax) attention

For queries q_i , keys k_j , values $v_j \in \mathbb{R}^d$ over a sequence of length n :

$$o_i = \frac{\sum_j \exp(q_i^\top k_j) v_j}{\sum_j \exp(q_i^\top k_j)}$$

- The exponential couples q_i and k_j *inside* the nonlinearity.
- Must materialize the $n \times n$ score matrix $QK^\top$.
- Cost: $\mathcal{O}(n^2d)$ time, $\mathcal{O}(n^2)$ memory (or $\mathcal{O}(n)$ KV cache at inference, growing with context).



The linearization trick

Replace the softmax kernel with a factorizable one:

$$\exp(\mathbf{q}^\top \mathbf{k}) \longrightarrow \phi(\mathbf{q})^\top \phi(\mathbf{k}) \quad \phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$$

Then

$$o_i = \frac{\phi(\mathbf{q}_i)^\top \sum_j \phi(\mathbf{k}_j) \mathbf{v}_j^\top}{\phi(\mathbf{q}_i)^\top \sum_j \phi(\mathbf{k}_j)} = \frac{\phi(\mathbf{q}_i)^\top \mathbf{S}}{\phi(\mathbf{q}_i)^\top \mathbf{z}}$$

- $\mathbf{S} = \sum_j \phi(\mathbf{k}_j) \mathbf{v}_j^\top \in \mathbb{R}^{d' \times d}$ and $\mathbf{z} = \sum_j \phi(\mathbf{k}_j)$ are **independent of i** .
- Compute them once, then each query costs $\mathcal{O}(d'd)$.

$\mathcal{O}(n d'd)$ — **linear in n** , memory independent of n .



Causal case: linear attention is an RNN

With a causal mask the sums become prefix sums:

$$S_t = S_{t-1} + \phi(k_t) v_t^\top$$

$$Z_t = Z_{t-1} + \phi(k_t)$$

$$o_t = \frac{\phi(q_t)^\top S_t}{\phi(q_t)^\top Z_t}$$

- A recurrent network with a **matrix-valued hidden state** $S_t \in \mathbb{R}^{d' \times d}$.
- Inference: $\mathcal{O}(1)$ time and memory per token — no KV cache.
- Katharopoulos et al. (2020), *"Transformers are RNNs."*



Choosing the feature map ϕ

Feature map	Notes
$\text{elu}(x) + 1$	Original; cheap; guarantees positivity
Positive random features	Performer; unbiased estimate of $\exp(q^\top k)$
Identity / ReLU / SiLU	Common in recent models
Drop the denominator	Replace with output normalization (LayerNorm / RMSNorm)

Most modern variants use a simple ϕ and normalize the output rather than the attention weights.



Why plain linear attention underperforms softmax

- **Bounded state.** S_t is $d' \times d$: fixed capacity regardless of context length. Retrieval of arbitrary past tokens degrades (needle-in-a-haystack, in-context recall).
- **Attention dilution.** Softmax yields sharp, near one-hot weights; kernel attention is smooth and spreads mass.
- **No forgetting.** $S_t = \sum_{j \leq t} \phi(k_j) v_j^\top$ only accumulates. Interfering keys blend their values and nothing is ever erased.



Linear Attention

d	Embedding Dim	2048
d_h	Q K V Dim	128
h, l	Heads, Layers	16

Component	Calculation	MParams (per layer)	MParams	Memory (MB)
K	$d_h \times d \times l$	0.26	4	16
Q	$d_h \times d \times h \times l$	4	67	256
V	$2 \times d_h \times d \times l$	0.52	8	32
Recurrent State	$(d_h^2 + d_h) \times l$	0.03	0.5	2

4 bytes at FP32, 2 bytes at FP16/BF16, 1 byte at FP8/INT8

Linear Attention

Limitations

- Sacrifice performance at the altar of efficiency.
- Storing context in a fixed-size recurrent state makes information retention harder.
- Unstable in training due to a lack of softmax-like normalization.

